

A Lightweight Convolutional Neural Network for Real-Time Recognition of Hand-Drawn Geometric Shapes

Shahir Abdullah

sshahnoor2620025@mscse.uiu.ac.bd

Abstract—Recognizing hand-drawn geometric shapes is a foundational sub-problem of sketch recognition, with applications in education, human-computer interaction, and diagram digitization. This paper presents the design, implementation, and evaluation of a desktop application that recognizes four basic hand-drawn geometric shapes, circle, square, rectangle, and triangle using a compact Convolutional Neural Network (CNN). A dataset of 2,000 labeled 28×28 -pixel shape images was collected independently and released publicly. The classifier consists of three convolutional blocks (16, 32, and 64 filters) with max-pooling, an in-model data-augmentation stage (random horizontal flip, rotation, and zoom), a dropout-regularized dense layer of 128 units, and a 4-way linear output layer, totaling 97,956 trainable parameters. The network is trained with the Adam optimizer on a sparse categorical cross-entropy objective computed directly on logits. On an 80/20 train-validation split, the model achieves 94.80% training accuracy and 96.01% validation accuracy with a validation loss of 0.1437. A Tkinter-based graphical interface allows a user to draw a shape with the mouse and receive an immediate class prediction with a confidence score. We situate this system within the broader sketch- and shape-recognition literature, compare its accuracy against related hand-drawn shape classification studies, and discuss the limitations inherent to a small, single-contributor dataset. The complete source code, trained model, and per-class datasets are released publicly to support reproducibility.

Index Terms—convolutional neural network, hand-drawn shape recognition, sketch recognition, image classification, deep learning, Tkinter, lightweight neural networks

I. INTRODUCTION

Sketching is one of the most natural ways for humans to externalize visual and spatial ideas, and geometric primitives. Geometric shapes such as circles, squares, rectangles, and triangles are the building blocks from which more complex hand-drawn diagrams, charts, and figures are composed. Automatically recognizing these primitives from freehand input has practical value for educational software that gives students instant feedback on geometry exercises, for note-digitization pipelines that convert whiteboard sketches into structured diagrams, and for sketch-based user interfaces more broadly.

While humans distinguish a circle from a triangle almost instantly and with near-perfect accuracy, teaching a machine to do the same from raw pixel input is non-trivial: strokes vary in thickness, closure, symmetry, and scale, and classical geometric descriptors are brittle to these variations. Convolutional Neural Networks (CNNs) address this by learning spatial-hierarchical features directly from pixel data, removing the

need for hand-engineered shape descriptors, and have become the default approach for small-image classification problems of this kind.

This paper documents a complete, self-contained pipeline for hand-drawn geometric shape recognition: a custom-collected dataset, a compact CNN trained on that dataset, and a working Tkinter desktop application that captures freehand mouse input and returns a real-time shape prediction with a confidence score. The specific contributions of this paper are:

- A custom dataset of 2,000 labeled, hand-drawn 28×28 images spanning four geometric classes (circle, square, rectangle, triangle), released publicly on Kaggle for reuse by other researchers.
- A lightweight CNN architecture, three convolutional blocks with 16, 32, and 64 filters, an in-model data-augmentation stage, and 97,956 trainable parameters in total that reaches 96.01% validation accuracy despite the small dataset size.
- A working, open-source graphical desktop application (Tkinter) that demonstrates end-to-end, real-time inference from freehand mouse input, including the exact preprocessing chain (grayscale conversion, resizing, edge enhancement) used at inference time.

The remainder of this paper is organized as follows. Section II reviews related work in classical shape recognition, sketch-recognition benchmarks, and CNN-based approaches to hand-drawn shape classification. Section III describes the dataset. Section IV details the proposed system, including the exact network architecture recovered from the released model checkpoint. Section V reports experimental results. Section VI compares these results against related studies. Section VII discusses the findings, Section VIII enumerates the limitations and threats to validity, and Section IX concludes with directions for future work.

II. RELATED WORK

A. Classical and Geometric Approaches

Prior to the widespread adoption of learned representations, shape recognition relied on geometric and contour-based descriptors. The Hough transform [6] detects parametric primitives such as lines and circles by accumulating votes in a parameter space, and remains a standard tool for detecting well-formed geometric primitives in clean line

drawings. Contour-based techniques approximate a shape’s outline with polygons or moment invariants and classify based on descriptors such as corner count, aspect ratio, or circularity. These methods can be effective on clean, noise-free input, but are sensitive to stroke gaps, self-intersections, and the irregularities typical of freehand sketching, which motivates learning-based alternatives.

B. Sketch-Recognition Datasets and Benchmarks

Eitz et al. [7] conducted the first large-scale study of human sketching behavior, collecting 20,000 sketches spanning 250 object categories (the TU-Berlin sketch dataset) and showing that even humans correctly classify unseen sketches only 73% of the time, while a bag-of-features representation with a multi-class SVM reached 56% accuracy. This result illustrates that general object-level sketch recognition is intrinsically difficult due to high intra-class variability. At a larger scale, Ha and Eck [8] introduced the QuickDraw-derived sketch-rnn framework, trained on millions of vector sketches collected through the Quick, Draw! game, and modeled sketches as sequences of pen strokes rather than static raster images, enabling both recognition and generative sketch modeling. These works establish that sketch recognition spans a spectrum from small, well-defined primitive sets (as addressed in this paper) to broad, open-vocabulary object recognition, with corresponding differences in achievable accuracy.

C. CNN-Based Recognition of Hand-Drawn Shapes

The use of small CNNs on low-resolution, grayscale-like images was established by LeCun et al. [9], whose LeNet architecture on $28 \times 28 / 32 \times 32$ digit images directly motivates the input resolution used in this work and in many subsequent small-image classification systems. Larger-scale CNNs such as AlexNet [10] later demonstrated that depth and parameter count can be scaled substantially for richer natural-image tasks, though such capacity is unnecessary for a four-class geometric primitive problem of the kind addressed here.

Most directly related to the present work, Alam et al. [11] proposed CnN-RFc, a hybrid CNN and random-forest classifier for hand-drawn mathematical geometric shapes, evaluated on a benchmark of 20,000 images across eight classes (circle, kite, parallelogram, square, rectangle, rhombus, trapezoid, triangle), reporting 98% accuracy with a Light Gradient Boosting Machine variant. Their work confirms that CNN-based feature extraction is well suited to hand-drawn geometric-shape classification and provides a useful accuracy reference point, albeit on a larger and more class-diverse benchmark than the one used here (Section VI).

Regularization techniques adopted in this work are similarly well established: dropout [12] randomly deactivates units during training to reduce co-adaptation and overfitting, and data augmentation surveyed broadly by Shorten and Khoshgof-taar [13] synthesizes label-preserving input variations (flips, rotations, zooms) to improve generalization when training data is limited, which is directly relevant given the modest size of the dataset used in this study.

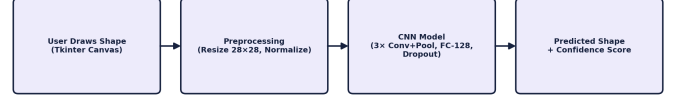


Fig. 1. End-to-end system pipeline: canvas input capture, preprocessing, CNN inference, and prediction output.

In summary, this work differs from the broader sketch-recognition literature in scope: rather than targeting large, open-vocabulary sketch categories (as in [7], [8]), it focuses on a minimal, well-defined set of four geometric primitives, using a deliberately lightweight CNN (under 100,000 parameters) suited to real-time CPU inference within a desktop GUI, rather than a server or GPU-class model.

III. DATASET

A. Data Collection

No public dataset of hand-drawn geometric shapes suited to this task was readily available at the time of the original project, so a dataset of 2,000 images spanning four classes, circle, square, rectangle, and triangle was created from scratch. Shapes were hand-drawn and compressed to a uniform resolution of 28×28 pixels, matching the input resolution conventions established by classic small-image benchmarks such as MNIST [9]. The per-class distribution of the 2,000 images is not recorded in the original project documentation and is therefore not reported here; we note this as a documentation gap rather than assert a class balance that cannot be verified.

B. Preprocessing and Splitting

The complete set of 2,000 images was split 80/20 into training and validation partitions using Keras’ image-preprocessing utilities. Pixel values are rescaled from the $[0, 255]$ range to $[0, 1]$ by a `Rescaling` layer (scale factor $1/255$) situated at the input of the network itself, and three data-augmentation layers `RandomFlip` (horizontal), `RandomRotation` (factor ± 0.1), and `RandomZoom` (height factor 0.1) are likewise embedded directly in the model graph and are active only during training, so that a single saved checkpoint contains the complete, reproducible preprocessing pipeline.

C. Dataset Availability

The four class-wise datasets were published on Kaggle [2]–[5] to support reuse by other researchers working on hand-drawn shape or sketch-recognition tasks.

IV. PROPOSED SYSTEM

A. System Overview

The system consists of three stages, illustrated in Fig. 1: (1) freehand shape capture through a drawing canvas, (2) a fixed preprocessing chain that converts the captured drawing into a normalized 28×28 input tensor, and (3) classification by a trained CNN that outputs one of four class labels together with a softmax confidence score.

TABLE I
NETWORK ARCHITECTURE AND PARAMETER COUNTS (TOTAL: 97,956
TRAINABLE PARAMETERS). *ACTIVE ONLY DURING TRAINING.

Layer	Output Shape	Params
Input	$28 \times 28 \times 3$	0
Data augmentation*	$28 \times 28 \times 3$	0
Rescaling ($\times 1/255$)	$28 \times 28 \times 3$	0
Conv2D (16, 3×3 , ReLU)	$28 \times 28 \times 16$	448
MaxPooling2D (2×2)	$14 \times 14 \times 16$	0
Conv2D (32, 3×3 , ReLU)	$14 \times 14 \times 32$	4,640
MaxPooling2D (2×2)	$7 \times 7 \times 32$	0
Conv2D (64, 3×3 , ReLU)	$7 \times 7 \times 64$	18,496
MaxPooling2D (2×2)	$3 \times 3 \times 64$	0
Dropout (rate = 0.2)	$3 \times 3 \times 64$	0
Flatten	576	0
Dense (128, ReLU)	128	73,856
Dense (4, linear / logits)	4	516

B. Drawing Interface and Input Capture

The front end is a desktop application built with Python’s Tkinter toolkit. It provides a 500×500 -pixel white canvas on which the user draws freehand with the mouse using a black pen of adjustable width (5 px by default); the in-memory PIL image mirrors every stroke drawn on the visible canvas. When the user requests a prediction, this 500×500 image is converted to a single-channel-plus-alpha (‘LA’) representation, resized to 28×28 pixels, and sharpened with PIL’s `EDGE_ENHANCE_MORE` filter before being written to disk and reloaded through Keras’ image-loading utility, which returns it as a 3-channel (RGB) array to match the network’s expected (28, 28, 3) input shape.

C. CNN Architecture

The exact architecture below was recovered directly from the released Keras checkpoint (`shapedetector_model_4b.h5`) rather than reconstructed from memory, and is reported here layer-by-layer for reproducibility. The classifier is a compact CNN consisting of three convolutional blocks (16, 32, and 64 filters respectively, each with 3×3 kernels, same padding, and ReLU activation), each followed by 2×2 max-pooling. The final pooled feature map ($3 \times 3 \times 64$) passes through a dropout layer (rate 0.2), is flattened to a 576-dimensional vector, and fed into a fully connected layer of 128 ReLU units before a final 4-unit linear (logit) output layer. Table I lists every layer, its output shape, and its parameter count.

D. Training Configuration

The network is trained with the Adam optimizer (learning rate = 0.001, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 1 \times 10^{-7}$), minimizing sparse categorical cross-entropy computed directly on the logits (`from_logits=True`) for numerical stability, with classification accuracy tracked as the reported metric. These values are taken directly from the training configuration embedded in the released checkpoint. The exact number of training epochs, batch size, and total training time were not recorded in the original project artifacts and are therefore not

TABLE II
FINAL TRAINING AND VALIDATION METRICS.

Metric	Value
Training Accuracy	94.80%
Validation Accuracy	96.01%
Validation Loss	0.1437
Trainable Parameters	97,956

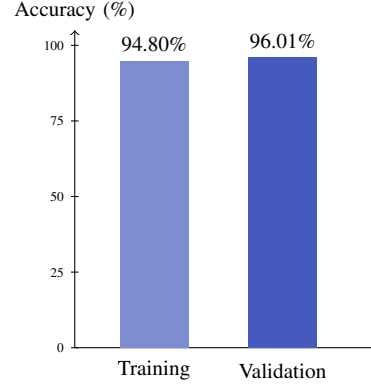


Fig. 2. Final training vs. validation accuracy.

reported; a fully reproducible re-run would require retraining with the released dataset and code under an explicitly logged configuration.

V. EXPERIMENTAL RESULTS

A. Evaluation Protocol

The model was evaluated on the 20% validation split held out from the 2,000-image dataset (Section III-B). No separate, independently held-out test set beyond this validation split is described in the original project documentation; this is noted explicitly as a limitation in Section VIII.

B. Quantitative Results

Table II summarizes the final training and validation metrics recorded for the model, and Fig. 2 visualizes the training-versus-validation accuracy.

C. Model Footprint

With only 97,956 trainable parameters, the network is several orders of magnitude smaller than large-scale image classifiers such as AlexNet [10] (approximately 60 million parameters), making it well suited to real-time CPU-only inference inside a desktop GUI without dedicated graphics hardware consistent with the sub-second prediction latency observed in the accompanying application.

VI. COMPARISON WITH RELATED WORK

Table III places the reported accuracy of this work alongside two related studies discussed in Section II. The comparison is illustrative rather than a controlled benchmark: the three studies differ substantially in class count, dataset size, and

TABLE III
ILLUSTRATIVE COMPARISON WITH RELATED HAND-DRAWN
SHAPE/SKETCH RECOGNITION STUDIES.

Study	Classes	Images	Accuracy
Eitz et al. [7], 2012	250 (general)	20,000	56% / 73% [†]
Alam et al. [11], 2025	8 (math. shapes)	20,000	98%
This work, 2021	4 (geometric)	2,000	96.01% (val.)

[†]computational / human accuracy, respectively.

task difficulty, so accuracy figures are not directly comparable and should be read as context rather than a ranked evaluation.

Two observations follow. First, the accuracy achieved here is broadly consistent with Alam et al. [11], supporting the view that hand-drawn geometric-primitive classification is a comparatively tractable sub-problem of the broader sketch-recognition space addressed in [7], where general object-category sketches are recognized correctly only 56-73% of the time even by humans and dedicated computer-vision pipelines. Second, this work reaches its accuracy with roughly an order of magnitude less training data and a far smaller network than [11], suggesting that for a minimal four-class geometric vocabulary, a lightweight CNN is sufficient without requiring a hybrid ensemble classifier.

VII. DISCUSSION

The validation accuracy (96.01%) exceeding the training accuracy (94.80%) is consistent with the regularization effect of the embedded dropout and data-augmentation layers (Section IV-C, D), which make the training-time task deliberately harder than plain inference on the (non-augmented) validation set a common and expected pattern when augmentation is applied only during training.

More broadly, the results show that a compact, sub-100,000-parameter CNN can achieve strong classification performance on a constrained, four-class shape-recognition problem even when trained on a modest, self-collected dataset of 2,000 images, without requiring transfer learning or a large pretrained backbone. This supports the use of small, task-specific CNNs for narrowly scoped sketch-recognition problems, in contrast to the larger architectures and datasets typically used for open-vocabulary sketch recognition (Section II-B).

VIII. LIMITATIONS AND THREATS TO VALIDITY

- **Single-contributor handwriting:** all 2,000 images were hand-drawn by one person, which may limit generalization to users with substantially different drawing styles, stroke widths, or input devices (e.g., touchscreen or stylus input, which was not evaluated).
- **No independent test set:** the original project documentation describes only an 80/20 train/validation split, with no separate held-out test set used solely for final reporting; the validation accuracy reported here may therefore be optimistic relative to unseen, out-of-distribution input.
- **Undocumented training configuration:** epoch count, batch size, and total training time are not recorded in

the released artifacts, limiting exact reproducibility of the training run (though the architecture and optimizer settings are fully recovered and reported in Section IV).

- **Narrow class scope:** the classifier distinguishes only four convex/simple primitives and has not been evaluated on additional polygons, overlapping shapes, incomplete strokes, or a rejection ('not a shape') class.
- **Unreported per-class metrics:** only aggregate accuracy and loss are available from the original project artifacts; a per-class confusion matrix was not recorded and is not fabricated here.

IX. CONCLUSION AND FUTURE WORK

This paper presented the design, exact architecture, and evaluation of a lightweight CNN-based system for recognizing hand-drawn geometric shapes, together with a working Tkinter desktop interface for real-time use. Recovered directly from the released model checkpoint, the network uses 97,956 trainable parameters across three convolutional blocks and a dense classification head, trained with Adam on sparse categorical cross-entropy, and reaches 96.01% validation accuracy on a self-collected, four-class, 2,000-image dataset.

Future work includes: expanding the label set to additional polygons (e.g., pentagons, hexagons, stars) and a negative/rejection class for non-shape scribbles; collecting drawings from multiple contributors, including touchscreen and stylus input, to improve generalization; introducing an explicit, held-out test set distinct from the validation split used for model selection; logging and publishing the full training configuration (epochs, batch size, hardware, wall-clock time) for exact reproducibility; and benchmarking lightweight alternative architectures (e.g., depthwise-separable convolutions) for further reductions in parameter count and inference latency on constrained or embedded devices.

REFERENCES

- [1] S. Abdullah, "Handwritten-Geometric-Shape-Detector," GitHub repository, 2021. [Online]. Available: <https://github.com/Shahir-Abdullah/Handwritten-Geometric-Shape-Detector>
- [2] S. Abdullah, "Handwritten Triangles Dataset," Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/shahirabdullah/handwritten-triangles>
- [3] S. Abdullah, "Handwritten Squares Dataset," Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/shahirabdullah/handwritten-squares>
- [4] S. Abdullah, "Handwritten Rectangles Dataset," Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/shahirabdullah/handwritten-rectangles>
- [5] S. Abdullah, "Handwritten Circles Dataset," Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/shahirabdullah/handwritten-circles>
- [6] R. O. Duda and P. E. Hart, "Use of the Hough transformation to detect lines and curves in pictures," *Commun. ACM*, vol. 15, no. 1, pp. 11–15, Jan. 1972.
- [7] M. Eitz, J. Hays, and M. Alexa, "How do humans sketch objects?," *ACM Trans. Graph. (Proc. SIGGRAPH)*, vol. 31, no. 4, pp. 44:1–44:10, 2012.
- [8] D. Ha and D. Eck, "A neural representation of sketch drawings," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2018.
- [9] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

- [10] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, 2012, pp. 1097–1105.
- [11] A. Alam, A. Raza, N. Thalji, L. Abualigah, H. Garay, J. Alemany-Iturriaga, and I. Ashraf, "Novel transfer learning approach for hand drawn mathematical geometric shapes classification," *PeerJ Comput. Sci.*, vol. 11, p. e2652, Jan. 2025.
- [12] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [13] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *J. Big Data*, vol. 6, no. 1, p. 60, 2019.
- [14] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.
- [15] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265–283.
- [16] F. Chollet *et al.*, *Keras*, 2015. [Online]. Available: <https://keras.io>
- [17] Python Software Foundation, "tkinter — Python interface to Tcl/Tk," Python 3 Documentation. [Online]. Available: <https://docs.python.org/3/library/tkinter.html>